PERTEMUAN 7 · Praktik terbimbing · Kuis konsep

Ambiguitas, Left Recursion, dan Left Factoring

Universitas Bale Bandung

Fakultas Teknologi Informasi · Program Studi Teknik Informatika

Mata Kuliah Teknik Kompilasi (MWCA017) · Semester Ganjil · 3 SKS

Dosen: Cecep Suwanda, S.Si., M.Kom.

Capaian Pertemuan

CPMK induk: CPMK 03 — Merancang tata bahasa (*grammar*) bebas konteks untuk bahasa sederhana.

Sub-CPMK yang dinilai pada pertemuan ini

Sub-CPMK 3.2: Menguji grammar subset bahasa C dan men-*debug* ambiguitas, *left recursion*, serta *left factoring* sampai berkas grammar bebas konflik.

Luaran yang diperiksa

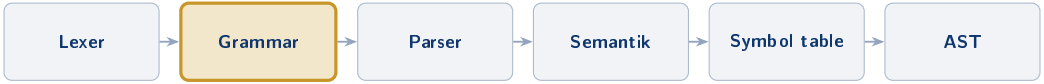
Berkas `grammar-111.ebnf` yang bebas konflik, log sebelum dan sesudah *debug*, lima string uji, dan bukti kesetaraan bahasa.

Agenda Pertemuan (150 menit)

Menit	Kegiatan
25	Ceramah: ambiguitas dan *dangling else*
20	Latihan: precedence, associativity, dan dua pohon urai
30	Latihan: menghapus *left recursion*
30	Latihan: *left factoring* dan konflik FIRST
20	Kuis konsep dan pembahasan
25	Praktik P2-B: uji dan *debug* grammar

Porsi praktik dan latihan 105 menit; kuis konsep 20 menit di tengah pertemuan.

Posisi Pertemuan pada Rantai Proyek



- Pertemuan ini membersihkan simpul **grammar** dari ambiguitas, rekursi kiri, dan prefiks umum.
- Grammar hasil transformasi menjadi syarat parser top-down pada Pertemuan 9.
- Berkas `grammar-ll1.ebnf` adalah artefak Sub-CPMK 3.2.

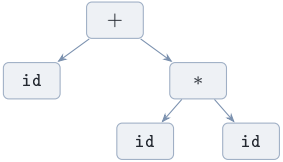
Ambiguitas Grammar

Definisi

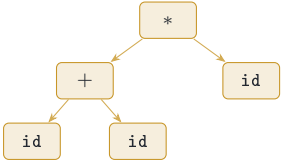
Tata bahasa G ambigu jika terdapat string $w \in L(G)$ yang memiliki dua pohon urai berbeda.

- Parser tidak dapat menentukan struktur yang dimaksud tanpa aturan tambahan.
- Contoh: $E \to E + E \mid E * E \mid \texttt{id}$.
- String $\texttt{id} + \texttt{id} * \texttt{id}$ memiliki dua pohon: satu mengutamakan $+$ di akar, satu mengutamakan $*$.
- Tanpa aturan prioritas, makna ekspresi tidak jelas.

Dua Pohon Urai untuk `id + id * id`



Pohon 1: akar $+$



Pohon 2: akar $*$

Keduanya valid menurut grammar. Aturan precedence memilih pohon 1 karena perkalian diikat lebih kuat.

Tiga Cara Menangani Ambiguitas

1. **Restrukturisasi grammar**: membuat tingkat prioritas terpisah.
2. **Aturan precedence dan associativity**: dipakai parser generator.
3. **Aturan semantik**: memilih pohon berdasarkan konteks, misalnya *else* terdekat.

Pilihan paling bersih

Restrukturisasi grammar menghilangkan ambiguitas pada tingkat sintaksis. Grammar aritmetika E, T, F adalah contohnya: satu nonterminal untuk satu tingkat prioritas.

Dangling Else

```
if (a)
    if (b) x;
    else y;       // else milik if(b)

if (a)
    if (b) x;
else y;           // else milik if(a)
```

Listing 1: Dua kemungkinan pemasangan `else`

- String: `if (a) if (b) x; else y;`.
- *Else* dapat dipasangkan ke `if` dalam atau luar.
- Aturan hampir semua bahasa: **else terdekat**.
- Ditulis dengan grammar *matched/unmatched* atau prioritas *shift* pada parser.
- Blok eksplisit memaksa pasangan luar.

Precedence dan Associativity

Tingkat	Operator	Associativity
1 (terendah)	`||`	kiri
2	`&&`	kiri
3	`==`, `!=`	kiri
4	`<`, `<=`, `>`, `>=`	kiri
5	`+`, `-`	kiri
6	`*`, `/`, `%`	kiri
7	`!`, unary `-`, unary `+`	kanan
8 (tertinggi)	`()`, call, literal, identifier	—

- Precedence: operator mana yang diikat lebih kuat.
- Associativity: pengelompokan pada precedence sama.
- Kiri: $a - b - c = (a - b) - c$.
- Kanan: $a = b = c = a = (b = c)$.
- Non-asosiatif: $a < b < c$ harus memakai kurung.

Menyatakan Precedence dalam Grammar

- Satu nonterminal untuk satu tingkat prioritas.
- Tingkat bawah menjadi anak dari tingkat atas.
- Pohon otomatis mencerminkan prioritas tanpa aturan tambahan.

Contoh Subset-C

additive → term {("+" | "-") term}.
`term` menangani perkalian, sehingga perkalian selalu lebih dalam.

```
%left OR
%left AND
%left EQ NE
%left LT LE GT GE
%left PLUS MINUS
%left STAR SLASH PERCENT
%right NOT UMINUS
```

Listing 2: Deklarasi precedence pada Bison

Left Recursion: Masalah dan Penghapusan

Mengapa bermasalah

Parser top-down memanggil fungsi untuk nonterminal. Aturan $A \to A\alpha$ membuat fungsi `parseA` memanggil dirinya sendiri sebelum mengonsumsi token, sehingga rekursi tak berujung.

Penghapusan langsung

$A \to A\alpha_1 \mid A\alpha_2 \mid \beta_1 \mid \beta_2$
menjadi
$A \to \beta_1 A' \mid \beta_2 A', \quad A' \to \alpha_1 A' \mid \alpha_2 A' \mid \varepsilon.$

A' menangani pengulangan; A memilih alternatif dasar.

Contoh: Menghapus *Left Recursion*

Ekspresi aritmetika

$E \to E + T \mid T$
menjadi
$E \to TE', \qquad E' \to +TE' \mid \varepsilon.$

Empat alternatif

$A \to Aa \mid Ab \mid c \mid d$
menjadi
$A \to cA' \mid dA', \qquad A' \to aA' \mid bA' \mid \varepsilon.$

Pohon urai tetap sama; parser dapat memutuskan berdasarkan token pertama.

Left Recursion Tidak Langsung dan Kondisi Nullable

Contoh tidak langsung

$A \to Ba \mid c$, $B \to Ab \mid d$.
Substitusi B ke A: $A \to Aba \mid da \mid c$.
Hapus langsung: $A \to daA' \mid cA'$, $A' \to baA' \mid \varepsilon$.

1. Urutkan nonterminal $A_1, \ldots, A_n$.
2. Untuk A_i, ganti aturan yang dimulai A_j $(j < i)$.
3. Hapus *left recursion* langsung pada A_i.

Jika α nullable, waspadai loop epsilon. Periksa FIRST/FOLLOW setelah transformasi.

Left Factoring

Transformasi

$A \to \alpha\beta_1 \mid \alpha\beta_2$
menjadi
$A \to \alpha A'$, $A' \to \beta_1 \mid \beta_2$. Jika salah satu alternatif berakhir setelah α, tambahkan ε pada A'.

If-else

$S \to \texttt{if}\ E\ \texttt{then}\ S\ \texttt{else}\ S \mid \texttt{if}\ E\ \texttt{then}\ S$
menjadi $S \to \texttt{if}\ E\ \texttt{then}\ S\ S'$, $S' \to \texttt{else}\ S \mid \varepsilon$.

Parser memutuskan setelah selesai memproses S; jika token berikutnya `else`, pilih alternatif pertama.

Contoh: Identifier dan Pemanggilan

Sebelum

primary $\rightarrow$ `ID` | `ID "(" args ")"`.

Sesudah

primary $\rightarrow$ `ID` primary$'$, primary$'$ $\rightarrow$ `"(" args ")"` | ε.

- Lexer membaca `ID` sekali.
- Parser memutuskan berdasarkan token berikutnya.
- Faktor umum pada Subset-C juga muncul pada `stmt` (assignment versus call).

Transformasi Grammar Subset-C

Aturan	Perubahan	Alasan
`primary`	`ID [ "(" [args] ")" ]`	menghilangkan prefiks `ID`
`stmt`	`ID assign_or_call ";"`	memisahkan assignment dan call
`expr`	tanpa assignment	menghindari konflik FIRST
`ifstmt`	`else` opsional	*dangling else* diselesaikan parser

Setelah factoring, grammar lebih cocok untuk parser LL(1). Grammar asli tetap disimpan untuk dokumentasi.

Berkas grammar-ll1.ebnf

```
4  // ekspresi tidak memuat assignment, dan primary
       memakai optional call.
5
6  program          = { topdecl } EOF ;
7  topdecl          = type ID "(" [ params ] ")" ( block
       | ";" ) ;
8  params           = param { "," param } ;
9  param            = type ID ;
10 type             = "int" | "float" | "char" | "void" ;
11 block            = "{" { stmt } "}" ;
12 stmt             = decl ";" | block | ifstmt |
       whilestmt | forstmt
13                  | returnstmt | breakstmt |
                        continuestmt | printstmt
14                  | ";" | ID assign_or_call ";" ;
15 assign_or_call   = "=" expr | "(" [ args ] ")" ;
16 decl             = type ID [ "=" expr ] ;
17 ifstmt           = "if" "(" expr ")" stmt [ "else"
       stmt ] ;
```

Listing 3: Bagian awal grammar-ll1.ebnf

- assign_or_call menangani = expr atau ([args]).
- stmt memakai satu prefiks ID.
- expr tidak lagi memuat assignment.
- Tingkat precedence dipertahankan.

Alur Uji dan *Debug* Grammar

Gejala	Penyebab	Tindakan
Dua pohon urai untuk satu string	Ambiguitas	Restrukturisasi atau aturan resolusi
Parser tidak pernah mengonsumsi token	*Left recursion*	Hapus langsung atau tidak langsung
Dua alternatif berebut token pertama	Prefiks umum	*Left factoring*
Sel ganda pada tabel LL(1)	Konflik FIRST/FOLLOW	Factoring, hapus rekursi, atau tambah lookahead
`else` masuk dua `if`	*Dangling else*	Aturan else terdekat atau grammar matched/unmatched

Uji lima string + satu kasus dangling else; terapkan satu transformasi pada satu waktu; uji ulang string yang sama.

Kuis Konsep

1. Apa definisi tata bahasa ambigu?
2. Mengapa *left recursion* menghambat parser top-down?
3. Ubah $A \to Aa \mid b$ menjadi bentuk tanpa *left recursion*.
4. Kapan *left factoring* diperlukan?
5. Apa aturan resolusi untuk *dangling else*?

Kerjakan mandiri selama 8 menit, lalu bahas bersama. Kunci tersedia pada versi handout.

Kegiatan Mahasiswa di Kelas: P2-B

Uji dan *debug* grammar

1. Mulai dari `spec/grammar.ebnf`.
2. Uji lima string, satu kasus dangling else, dan satu ekspresi campuran; catat konflik.
3. Debug satu masalah pada satu waktu: ambiguitas, rekursi kiri, lalu factoring.
4. Tulis hasil ke `spec/grammar-lll.ebnf`.
5. Uji ulang string yang sama; buktikan kesetaraan bahasa.
6. Kumpulkan log sebelum dan sesudah debug.

Tugas dan Artefak

Artefak Sub-CPMK 3.2

Berkas grammar yang sudah dibebaskan dari ambiguitas, *left recursion*, dan kebutuhan *left factoring*, disertai log sebelum/sesudah dan bukti uji.

- Tidak ada *left recursion* langsung maupun tidak langsung.
- Prefiks umum sudah difaktorkan; konflik FIRST hilang.
- *Dangling else* memiliki aturan resolusi eksplisit.
- Lima string uji tetap valid; kasus dangling else benar.
- `grammar-ll1.ebnf` dan log sebelum/sesudah dikumpulkan.

P2-B menjadi bagian Praktikum 2 (12%) yang dilaporkan pada Pertemuan 9.

Rangkuman dan Jembatan

- Ambiguitas berarti satu string memiliki dua pohon urai; diselesaikan dengan restrukturisasi, precedence, atau aturan semantik.
- Precedence dan associativity ditentukan oleh tingkat nonterminal dan urutan aturan.
- *Left recursion* dihapus dengan nonterminal bantu A' yang menangani pengulangan.
- *Left factoring* mengeluarkan prefiks umum agar parser dapat memutuskan dengan satu token lookahead.
- Grammar bebas konflik dibuktikan dengan lima string uji, kasus dangling else, dan log sebelum/sesudah.

Pertemuan berikutnya

Pertemuan 9 memakai grammar yang bersih untuk menghitung FIRST dan FOLLOW, memutuskan LL(1), dan membangun recursive descent parser.